

Resenha: Design by Contract – Bertrand Meyer

O capítulo “Design by Contract”, escrito por Bertrand Meyer, apresenta uma das ideias mais influentes da engenharia de software orientada a objetos: o conceito de contratos formais entre componentes de software. A proposta central do autor é tratar o desenvolvimento de sistemas como uma relação contratual, na qual cada módulo possui obrigações e garantias claramente definidas. Essa abordagem busca aumentar a confiabilidade, robustez e clareza dos sistemas, reduzindo ambiguidades na interação entre classes, rotinas e usuários do software.

Meyer inicia o texto destacando um problema recorrente no desenvolvimento de software: a dificuldade de construir sistemas corretos e resistentes a falhas. Segundo ele, muitas abordagens tradicionais focavam apenas em organização estrutural e reutilização de código, mas deixavam em segundo plano questões relacionadas à correção e confiabilidade. Nesse contexto, o Design by Contract surge como uma metodologia capaz de formalizar responsabilidades entre diferentes partes do sistema.

A metáfora do contrato é utilizada para explicar o funcionamento da proposta. Assim como em contratos do mundo real, cada lado possui deveres e benefícios. Em software, o “cliente” é quem utiliza uma rotina, enquanto o “fornecedor” é a rotina ou classe responsável pela operação. O cliente deve garantir certas condições antes de realizar a chamada da rotina; em troca, o fornecedor garante determinados resultados após a execução.

Essas condições são representadas pelas chamadas assertions (asserções). Meyer divide as asserções em três categorias principais: preconditions, postconditions e class invariants. As preconditions definem o que deve ser verdadeiro antes da execução de uma rotina. Já as postconditions especificam o estado esperado após a execução. Os invariantes de classe representam propriedades que devem permanecer verdadeiras durante todo o ciclo de vida do objeto.

Um dos exemplos apresentados no texto é o de uma estrutura de tabela com uma operação de inserção. A rotina de inserção exige, como pré-condição, que a tabela não esteja cheia. Em troca, garante como pós-condição que o novo elemento foi inserido corretamente e que o número de itens aumentou. Esse modelo deixa explícitas as responsabilidades de cada parte e evita verificações redundantes.

A crítica de Meyer ao defensive programming é um dos pontos mais marcantes do capítulo. Tradicionalmente, muitos programadores adicionam verificações excessivas dentro das rotinas, tentando tratar qualquer possível erro. Para o autor, isso aumenta a complexidade e reduz a clareza do sistema. No modelo de contratos, cada condição deve possuir apenas um responsável. Se determinada verificação é obrigação do cliente, a rotina não precisa repeti-la internamente.

Outro aspecto importante discutido é o tratamento de exceções. Meyer argumenta que exceções não devem ser utilizadas como substitutas para estruturas normais de controle, mas sim como mecanismos para lidar com falhas reais no cumprimento de contratos. O autor critica fortemente o uso inadequado de exceções em linguagens como Ada e CLU, mostrando exemplos em que o programa continua executando mesmo após falhas graves.

A abordagem proposta no texto estabelece três princípios fundamentais para tratamento de exceções: a rotina deve cumprir seu contrato ou falhar explicitamente; falhas devem gerar exceções perceptíveis ao cliente; e, ao ocorrer uma exceção, a rotina deve restaurar o sistema para um estado consistente antes de tentar novamente ou encerrar a execução. Essa visão torna o mecanismo de exceções mais disciplinado e previsível.

Meyer também relaciona o conceito de contratos à herança e ao polimorfismo. Em herança, subclasses podem redefinir métodos, mas precisam respeitar as condições do contrato original. Isso significa que uma subclasse não pode exigir pré-condições mais fortes nem oferecer pós-condições mais fracas. Essa regra garante que o polimorfismo funcione de forma segura, mantendo a compatibilidade entre classes base e derivadas.

Outro ponto relevante é a valorização de funções parciais. O autor defende que nem toda rotina precisa funcionar em qualquer situação possível. Em muitos casos, tentar tratar todos os cenários leva a sistemas extremamente complexos. Para Meyer, é melhor criar componentes simples, claros e bem documentados, deixando explícitas suas limitações através das pré-condições.

O texto também destaca a importância das assertions como ferramenta de documentação e depuração. Em vez de depender apenas de comentários informais, o contrato formal serve como especificação precisa do comportamento esperado. Além disso, violações de contratos ajudam a localizar bugs rapidamente, indicando se o problema ocorreu no cliente ou no fornecedor da operação.

Conclusão

“Design by Contract” apresenta uma abordagem extremamente relevante para a engenharia de software moderna. A proposta de Bertrand Meyer vai além de simples técnicas de programação, oferecendo uma filosofia de construção de sistemas baseada em clareza, responsabilidade e confiabilidade. Seus conceitos influenciaram práticas posteriores, como Clean Architecture, Domain-Driven Design e metodologias de desenvolvimento orientadas a testes.

Apesar de exigir maior disciplina dos desenvolvedores, o uso de contratos formais contribui para a criação de sistemas mais organizados, previsíveis e fáceis de manter. O texto demonstra que robustez não depende de verificações excessivas, mas de responsabilidades bem definidas entre os componentes do software. Dessa forma, o Design by Contract permanece como uma das contribuições mais importantes para a teoria e prática da programação orientada a objetos.